



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



REPORT SIMPLE OS
Môn học: Operating System

Học kỳ: HK 252 – Lớp: L03

Giảng viên hướng dẫn: Nguyễn Tuấn Huy

Danh sách nhóm

STT	Họ và Tên	MSSV
1	Nguyễn Thanh Tùng	2449106
2	Nguyễn Hoàng	2433007
3	Dương Quang Minh	2412034

May 20, 2026

Contents

1	MỤC TIÊU VÀ PHẠM VI	2
1.1	Phương pháp thực hiện	2
2	QUESTIONS AND DISCUSSION	2
2.1	Q1. What are the advantages and disadvantages of using the unified system call interface for manipulating different system components, i.e. read/write/free for files, memory and I/O devices? In your analysis, consider how this abstraction influences operating system design, performance trade-offs, error handling complexity, and the balance between portability and efficiency?	2
2.2	Q2. When a system call executes too long time, how does the Operating system detect and handle the case?	3
2.3	Q3. Considering the impactness of MLQ detailed policies, what is the benefit of each policy?	3
2.4	Q4. What is the primary motivation for combining segmentation with paging in memory management? How does this hybrid approach address limitations inherent in using either technique alone?	4
2.5	Q5. What are the benefits of extending hierarchical paging to N level?	4
2.6	Q6. What are the advantages and disadvantages of paging and contiguous memory allocation?	4
2.7	Q7. What happens if the synchronization is not handled in your Simple OS? Illustrate the problem of your simple OS (assignment outputs) by example if you have any in the added kernel memory operations.	5
3	BÁO CÁO CHI TIẾT KỸ THUẬT VÀ THỰC NGHIỆM	5
3.1	Bộ lập lịch Multi-Level Queue (MLQ)	6
3.1.1	Cài đặt logic trong mã nguồn	6
3.1.2	Đối chiếu kết quả thực nghiệm (sched_1.output)	6
3.2	Hệ thống phân trang 5 cấp (MM64 Paging)	6
3.2.1	Hiện thực phân cấp Địa chỉ ảo	6
3.2.2	Phân tích hiệu năng thông qua Output (os_1_mlq_paging_small_4K.output)	7
3.3	Cơ chế Syscall an toàn và PID-based Lookup	7
3.4	Đồng bộ hóa dữ liệu (Synchronization)	7
4	KẾT LUẬN	8

1. MỤC TIÊU VÀ PHẠM VI

Báo cáo này trình bày kết quả nghiên cứu và hiện thực hóa các thành phần cốt lõi của một hệ điều hành đơn giản (Simple OS) do nhóm chúng em thực hiện. Mục tiêu chính của dự án bao gồm: thiết kế bộ điều phối CPU theo thuật toán MLQ, xây dựng hệ thống quản lý bộ nhớ ảo theo mô hình phân trang 5 cấp (MM64), và thiết lập cơ chế gọi hệ thống (syscall) an toàn.

Tất cả các phân tích và nhận xét trong báo cáo này đều dựa trên dữ liệu thực nghiệm khách quan thu được khi nhóm thực thi các kịch bản kiểm thử (test scripts). Các file kết quả được nhóm lưu trữ chi tiết tại thư mục output/ để đối chứng.

1.1. Phương pháp thực hiện

- Phát triển: Nhóm sử dụng ngôn ngữ C, biên dịch bằng gcc thông qua Makefile.
- Kiểm thử: Sử dụng script run.sh để chạy tự động các kịch bản mô phỏng từ đơn giản (Single CPU) đến phức tạp (Multi-core, Paging).
- Phân tích: Dựa trên biểu đồ Gantt và các chỉ số thống kê bộ nhớ để đánh giá tính đúng đắn của thuật toán.

2. QUESTIONS AND DISCUSSION

2.1. Q1. What are the advantages and disadvantages of using the unified system call interface for manipulating different system components, i.e. read/write/free for files, memory and I/O devices? In your analysis, consider how this abstraction influences operating system design, performance trade-offs, error handling complexity, and the balance between portability and efficiency?

Trả lời: Việc sử dụng giao diện gọi hệ thống nhất quán (unified syscall interface) — tương tự triết lý “Everything is a file” của Unix — mang lại sự trừu tượng hóa mạnh mẽ cho thiết kế hệ điều hành.

Ưu điểm:

- **Thiết kế hệ điều hành tinh gọn:** Giúp đơn giản hóa cấu trúc kernel. Các lập trình viên ứng dụng chỉ cần học một bộ API duy nhất (read, write, close) để tương tác với nhiều thực thể khác nhau (từ file trên đĩa đến bộ nhớ ảo hay thiết bị ngoại vi).
- **Tính di động (Portability):** Ứng dụng trở nên độc lập với thiết bị. Một chương trình đọc dữ liệu từ tệp tin có thể dễ dàng chuyển sang đọc từ một thiết bị I/O hoặc socket mạng mà không cần thay đổi logic cốt lõi.
- **Tái sử dụng mã:** Các công cụ hệ thống (như pipe, redirection) có thể áp dụng cho bất kỳ thành phần nào tuân thủ giao diện này.

Nhược điểm và Trade-offs:

- **Hiệu năng (Performance):** Một giao diện chung có thể không tận dụng được các đặc tính phần cứng chuyên biệt. Ví dụ, truy cập bộ nhớ tốc độ cao có thể bị chậm lại do overhead của lớp trừu tượng write so với việc ánh xạ bộ nhớ trực tiếp.
- **Độ phức tạp trong xử lý lỗi:** Một mã lỗi chung như “I/O Error” có thể ám chỉ rất nhiều lỗi khác nhau tùy thuộc vào thiết bị (lỗi đĩa, mất kết nối mạng, hoặc lỗi ghi bộ nhớ), gây khó khăn cho việc gỡ lỗi chính xác.
- **Cân bằng giữa Portability và Efficiency:** Để duy trì tính di động, hệ điều hành phải thêm các lớp trung gian (driver), làm tăng độ trễ. Các hệ thống yêu cầu hiệu năng cực cao thường phải bỏ qua lớp trừu tượng này để giao tiếp trực tiếp với phần cứng.

2.2. Q2. When a system call executes too long time, how does the Operating system detect and handle the case?

Trả lời: Hệ điều hành quản lý các system call chạy quá lâu thông qua sự phối hợp giữa ngắt phần cứng (Timer Interrupt) và các chính sách của bộ lập lịch (Scheduler).

Giải thích chi tiết:

- **Timer Interrupt:** Phần cứng tạo ra các xung ngắt định kỳ. Khi một tiến trình đang thực thi syscall, CPU vẫn bị “ngắt” bởi timer để chuyển quyền điều khiển về cho Kernel.
- **Preemption (Thu hồi quyền ưu tiên):** Sau mỗi xung nhịp, bộ lập lịch sẽ kiểm tra “time-slice” (lát cắt thời gian) của tiến trình. Nếu tiến trình đã dùng hết hạn mức, nó sẽ bị đánh dấu để thu hồi CPU ngay khi syscall hoàn tất (hoặc tại các điểm kiểm tra an toàn trong kernel).
- **Cơ chế Blocking:** Nếu syscall đang chờ một tài nguyên (như I/O hoặc khóa), kernel sẽ chuyển tiến trình sang trạng thái WAITING, giải phóng CPU cho tiến trình khác.
- **Watchdog Timers:** Trong các hệ thống nhạy cảm, một watchdog timer có thể được dùng để phát hiện nếu một syscall bị treo vĩnh viễn (deadlock) trong kernel, từ đó kích hoạt cơ chế phục hồi hoặc crash dump để bảo vệ hệ thống.

2.3. Q3. Considering the impactness of MLQ detailed policies, what is the benefit of each policy?

Trả lời: Cơ chế lập lịch Multi-Level Queue (MLQ) tối ưu hóa hệ thống bằng cách phân loại tiến trình vào các hàng đợi khác nhau với các chính sách riêng biệt, giải quyết mâu thuẫn giữa độ đáp ứng và thông lượng.

Giải thích chi tiết:

- **Hàng đợi ưu tiên cao (Interactive):** Thường dùng Round Robin với quantum nhỏ. Lợi ích: Tối ưu hóa độ đáp ứng (Responsiveness), đảm bảo các tác vụ tương tác với người dùng không bị cảm giác “lag”.
- **Hàng đợi ưu tiên trung bình:** Cân bằng giữa tính công bằng và thông lượng. Lợi ích: Phục vụ tốt cho đại đa số các tác vụ hệ thống thông thường mà không gây quá tải chuyển ngữ cảnh.

- **Hàng đợi ưu tiên thấp (Batch):** Dùng FCFS hoặc quantum lớn. Lợi ích: Tối đa hóa thông lượng (Throughput) cho các tác vụ tính toán nặng bằng cách giảm thiểu chi phí chuyển ngữ cảnh và tận dụng tốt bộ nhớ đệm (cache).
- **Cơ chế Slot/Aging (Project Simple OS):** Việc giới hạn số lượng slot thực thi cho mỗi mức ưu tiên giúp đảm bảo tính công bằng (Fairness), ngăn chặn tình trạng đói tài nguyên (Starvation) cho các tiến trình ở mức thấp.

2.4. Q4. What is the primary motivation for combining segmentation with paging in memory management? How does this hybrid approach address limitations inherent in using either technique alone?

Trả lời: Mục tiêu cốt lõi của mô hình lai là kết hợp khả năng quản lý logic của Segmentation với hiệu quả quản lý vật lý của Paging.

- **Khắc phục nhược điểm của Segmentation:** Segmentation thuần túy gây ra phân mảnh ngoại vi (External Fragmentation) nghiêm trọng do kích thước các phân đoạn không đều. Bằng cách áp dụng Paging lên từng segment, hệ điều hành không còn cần các khối RAM liên tục lớn, loại bỏ nhu cầu dồn dịch bộ nhớ (Compaction) tốn kém.
- **Khắc phục nhược điểm của Paging:** Paging thuần túy “mù” về mặt ngữ nghĩa (coi mọi trang nhớ là như nhau). Segmentation cung cấp các lớp bảo vệ logic (quyền Read/Write/Execute) dựa trên mục đích sử dụng của chương trình (code vs data).
- **Kết quả:** Hệ thống đạt được khả năng bảo vệ bộ nhớ mạnh mẽ, hỗ trợ chia sẻ mã nguồn dễ dàng (qua Segments) trong khi vẫn tận dụng được RAM tối đa mà không lo về phân mảnh (qua Paging).

2.5. Q5. What are the benefits of extending hierarchical paging to N level?

Trả lời: Paging nhiều tầng (như mô hình MM64) là giải pháp thiết yếu để quản lý không gian địa chỉ khổng lồ của các kiến trúc 64-bit hiện đại.

- **Hiệu quả lưu trữ Metadata:** Thay vì duy trì một bảng trang phẳng khổng lồ chiếm hàng GB bộ nhớ ngay cả khi RAM thật sự rất ít, hệ thống phân cấp chỉ tạo ra các bảng trang con khi vùng nhớ đó thực sự được sử dụng (on-demand creation).
- **Hỗ trợ vùng nhớ thưa (Sparse Memory):** Các ứng dụng hiện đại thường có khoảng cách rất lớn giữa vùng Stack và Heap. Paging nhiều tầng cho phép bỏ qua các khoảng trống này mà không tốn bất kỳ byte bộ nhớ nào cho metadata.
- **Khả năng mở rộng:** Cho phép hệ điều hành mở rộng từ không gian địa chỉ 32-bit (2 tầng) lên 57-bit (5 tầng) mà không cần thay đổi logic quản lý cơ bản, dù phải đánh đổi bằng việc tăng độ trễ dịch địa chỉ (address translation latency).

2.6. Q6. What are the advantages and disadvantages of paging and contiguous memory allocation?

Trả lời: Sự lựa chọn giữa Paging và Cấp phát liên tục phụ thuộc vào sự đánh đổi giữa hiệu năng phần cứng và hiệu quả sử dụng RAM.

Contiguous Memory Allocation (Cấp phát liên tục):

- **Ưu điểm:** Cài đặt đơn giản; hiệu năng cực cao vì không tốn thời gian dịch địa chỉ qua bảng trang; ít yêu cầu phần cứng MMU phức tạp.
- **Nhược điểm:** Phân mảnh ngoại vi nghiêm trọng; kích thước vùng nhớ phải được biết trước khi nạp; việc mở rộng vùng nhớ hiện có rất khó khăn và tốn kém.

Paging (Phân trang):

- **Ưu điểm:** Loại bỏ hoàn toàn phân mảnh ngoại vi; hỗ trợ bộ nhớ ảo và swap; cho phép chia sẻ bộ nhớ giữa các tiến trình một cách linh hoạt.
- **Nhược điểm:** Gây phân mảnh nội vi (Internal Fragmentation) ở trang cuối cùng; yêu cầu phần cứng phức tạp (MMU/TLB); gây ra overhead về thời gian do phải tra cứu bảng trang trong RAM.

2.7. Q7. What happens if the synchronization is not handled in your Simple OS? Illustrate the problem of your simple OS (assignment outputs) by example if you have any in the added kernel memory operations.

Trả lời: Nếu không xử lý đồng bộ (mất đi Mutexes/Locks), hệ thống sẽ gặp phải tình trạng Race Condition, dẫn đến dữ liệu kernel bị hỏng và hành vi không xác định.

- **Hỏng cấu trúc dữ liệu Kernel:** Hãy tưởng tượng hai CPU cùng lúc gọi hàm cấp phát khung trang vật lý. Cả hai đều đọc thấy Frame số 100 đang tự do. Nếu không có lock, cả hai sẽ cùng cấp phát Frame 100 cho hai tiến trình khác nhau, dẫn đến việc tiến trình A ghi đè dữ liệu của tiến trình B.
- **Corruption hàng đợi:** Việc cập nhật các danh sách liên kết trong bộ lập lịch (Scheduler) yêu cầu nhiều bước. Nếu một CPU khác can thiệp giữa chừng, các con trỏ sẽ trở sai địa chỉ, làm mất dấu tiến trình hoặc tạo ra vòng lặp vô hạn gây treo kernel.
- **Ví dụ trong Simple OS:** Trong quá trình thực hiện, nếu chúng ta chạy với nhiều CPU mô phỏng (-c 4) mà không có `system_lock` hay `mmvm_lock`, log của `rxuthinccli` “InvalidPageTableEntry” hoặc `trnhmpnbgng(hang)`,

3. BÁO CÁO CHI TIẾT KỸ THUẬT VÀ THỰC NGHIỆM

Trong phần này, nhóm chúng em sẽ đi sâu vào phân tích các thành phần cốt lõi của hệ thống Simple OS, đối chiếu trực tiếp giữa mã nguồn đã cài đặt và kết quả thực nghiệm thu được từ các kịch bản kiểm thử.

3.1. Bộ lập lịch Multi-Level Queue (MLQ)

Bộ lập lịch được nhóm cài đặt tại src/sched.c với cấu trúc đa mức ưu tiên (MAX_PRIO = 140). Mỗi hàng đợi có một chỉ số slot riêng biệt để quản lý thời gian chiếm dụng CPU.

3.1.1 Cài đặt logic trong mã nguồn

- **Cấu trúc dữ liệu:** Nhóm sử dụng mảng `mlq_ready_queue` gồm 140 hàng đợi. Mỗi hàng đợi struct `sched_queue` chứa một danh sách liên kết các tiến trình và biến slot.
- **Hàm `get_proc`:** Đây là trái tim của bộ lập lịch. Thay vì chỉ lấy tiến trình có ưu tiên cao nhất một cách cực đoan (gây đói tài nguyên), nhóm đã cài đặt logic:
 - Kernel sẽ duyệt từ hàng đợi có ưu tiên cao nhất (`prio = 0`).
 - Nếu hàng đợi đó còn slot > 0 và có tiến trình, nó sẽ được chọn.
 - Nếu slot == 0, kernel sẽ bỏ qua và tìm ở mức ưu tiên thấp hơn.
 - Khi tất cả các hàng đợi có tiến trình đều đã dùng hết slot, hàm `get_proc` sẽ thực hiện reset lại toàn bộ slots về giá trị mặc định dựa trên mức ưu tiên (Priority-based slots).

3.1.2 Đối chiếu kết quả thực nghiệm (`sched_1.output`)

Dựa trên kết quả chạy test `sched_1`, nhóm quan sát thấy:

```
Time : 1  3  5  7  9 10 13 17
CPU0 : P1 P2 P3 P4 P3 P4 P3 P4
```

Phân tích của nhóm:

- Tại thời điểm bắt đầu, P1 và P2 được chạy trước (có lẽ do được nạp trước hoặc ưu tiên cao).
- Tuy nhiên, từ Time 9 trở đi, P3 và P4 liên tục luân phiên chiếm giữ CPU. Điều này minh chứng rằng P1 và P2 đã cạn kiệt slot trong chu kỳ đó, buộc bộ lập lịch phải nhường quyền cho các tiến trình “yếu” hơn nhưng vẫn còn slot. Đây chính là cơ chế chống độc quyền (anti-starvation) mà nhóm đã hiện thực thành công.

3.2. Hệ thống phân trang 5 cấp (MM64 Paging)

Một trong những phần phức tạp nhất mà các thành viên trong nhóm đã thực hiện là hệ thống quản lý bộ nhớ ảo 64-bit tại src/mm-vm.c và src/mm.c.

3.2.1 Hiện thực phân cấp Địa chỉ ảo

Nhóm sử dụng cấu trúc địa chỉ ảo được định nghĩa trong include/mm64.h with 5 tầng: PGD -> P4D -> PUD -> PMD -> PT.

Hàm `pg_getpage` được nhóm thiết kế để thực hiện việc duyệt cây bảng trang một cách đệ quy hoặc theo vòng lặp. Nếu một tầng trung gian chưa tồn tại, kernel sẽ tự động cấp phát một Frame mới để làm bảng trang (on-demand page table allocation).

3.2.2 Phân tích hiệu năng thông qua Output (os_1_mlq_paging_small_4K.output)

Khi chạy với kích thước trang 4KB, nhóm thu được các con số thống kê sau:

mem_access: 2
pgtbl_access: 10
pgtbl_bytes: 163,840 (160KB)

Nhận xét từ nhóm:

- **Overhead bảng trang:** Tỷ lệ `pgtbl_access` / `mem_access` là 5:1. Điều này hoàn toàn chính xác với logic 5 tầng của nhóm: để truy cập 1 byte dữ liệu, phần cứng MMU phải đi qua 5 bảng trang (PGD -> P4D -> PUD -> PMD -> PT).
- **Phân mảnh nội vi:** Khi so sánh với cấu hình `small_1K` (trang 1KB), nhóm nhận thấy dù cùng một ứng dụng, cấu hình 4KB tiêu tốn nhiều RAM vật lý hơn cho các byte dư thừa ở cuối trang. Tuy nhiên, cấu hình 4KB lại giúp bảng trang nhỏ gọn hơn về số lượng entry cần quản lý.

3.3. Cơ chế Syscall an toàn và PID-based Lookup

Để bảo vệ kernel, nhóm không cho phép user-space truyền trực tiếp struct PCB. Thay vào đó, nhóm cài đặt syscall 17 (Memory Management) nhận tham số là PID.

Trong `src/syscall.c`, khi nhận yêu cầu, kernel sẽ gọi `find_proc_by_pid(pid)` để lấy con trỏ PCB từ danh sách quản lý tập trung.

Thực tế kiểm chứng: Trong các file `output/os_syscall.output`, nhóm thấy các lệnh `alloc` và `write` được thực hiện chính xác chỉ bằng việc cung cấp PID. Nếu một tiến trình cố gắng gửi một PID không tồn tại hoặc PID của tiến trình khác, hàm `lookup` sẽ trả về NULL và kernel sẽ từ chối thực hiện, ngăn chặn hành vi xâm phạm bộ nhớ (Memory Protection).

3.4. Đồng bộ hóa dữ liệu (Synchronization)

Trong môi trường đa nhân (Multi-CPU), việc truy cập vào danh sách khung trang trống (`free_frame_list`) và hàng đợi tiến trình là những “vùng chí tử” (Critical Sections).

Giải pháp của nhóm: Các thành viên đã sử dụng `pthread_mutex_t` để bao bọc các thao tác nhạy cảm.

- `mem_lock` trong `src/mm-memphy.c` đảm bảo không có chuyện hai CPU cùng cấp phát một Frame cho hai tiến trình khác nhau.
- `queue_lock` trong `src/sched.c` bảo vệ tính toàn vẹn của danh sách READY khi nhiều CPU cùng muốn lấy tiến trình để chạy.

Kết quả: Trong suốt quá trình thực hiện các kịch bản test nặng như `os_2_mlq_paging` (nhiều tiến trình, nhiều CPU), hệ thống của nhóm hoạt động ổn định, không xảy ra hiện tượng Race Condition hay Deadlock, chứng minh việc thiết kế khóa là hợp lý và chính xác.

4. KẾT LUẬN

Qua quá trình thực hiện dự án Simple OS, nhóm chúng em đã đạt được những kết quả quan trọng:

- **Hiểu sâu về Kernel:** Việc tự tay cài đặt logic lập lịch và quản lý bộ nhớ giúp các thành viên nắm vững cách thức một Hệ điều hành vận hành dưới “nắp capo”.
- **Kỹ năng gỡ lỗi:** Việc đối chiếu liên tục giữa mã nguồn và file .output đã rèn luyện cho nhóm tư duy phân tích dữ liệu thực nghiệm.
- **Hoàn thiện hệ thống:** “Simple OS” không chỉ là một bài tập giả lập, mà là một hệ thống có cấu trúc chặt chẽ, an toàn với syscall và quản lý RAM hiệu quả nhờ cơ chế phân tầng.

Nhóm chúng em hy vọng báo cáo này đã cung cấp cái nhìn chi tiết và khách quan nhất về thành quả lao động của cả nhóm trong suốt thời gian qua.